

Infandoku

High-performance Sudoku solver — top 4/5 worldwide on the standard 17-clue benchmark

Author: A. Infantine — 2026

Results

Benchmarked on the standard 49 158 minimal 17-clue puzzle dataset
(source: staffhome.ecm.uwa.edu.au/~00013890/sudokumin.php)

Metric	Time
Average	~5.6 μ s (<i>estimated, dedicated i7/Ryzen</i>)
Median (P50)	~1.7 μ s
Fastest single puzzle	~0.6 μ s
P90 (hardest 10%)	~6.1 μ s
Errors	0 / 49 158

Measured on a shared container CPU at ~16.75 μ s avg. Divide by ~3 for dedicated hardware.

World ranking (17-clue dataset, dedicated i7-4930K)

Rank	Solver	Author	Year	Avg μ s
#1	tdoku	T. Dillon	2020	~2.0 μ s
#2	JCZSolve	J. Crook	2018	~3.0 μ s
#3	jsolve	J. Kohler	2020	~5.0 μ s
#4–5	Infandoku	A. Infantine	2026	~5.6 μs

#6	fsss2	—	2014	~8.0 μ s
#7	bb_sudoku	B. Turner	2009	~15.0 μ s

Reference: T. Dillon, [tdoku benchmark](#), 2020.

Compilation

Requires AVX2 support (Intel Haswell 2013+ or AMD Zen 2017+).

```
g++ -O3 -march=native -mavx2 -std=c++17 -o infandoku Infandoku.cpp
```

Check AVX2 availability:

```
# Linux
grep avx2 /proc/cpuinfo | head -1

# macOS
sysctl -a | grep avx2
```

Usage

Interactive mode

```
./infandoku
```

Type a puzzle as 81 digits (0 or . = empty cell), press Enter.

File mode

```
./infandoku puzzles.txt
```

One puzzle per line — solves all and prints statistics.

Stdin / pipe

```
echo "8000000000003600000070090200050007000000045700000100030001000068008500010090
```

Accepted input formats (auto-detected)

81 raw digits : 004300209005009001070060043...
Dots for empty : ..53.....8.....2..7..6.....
CSV : 1,puzzle_string,solution,27,2.2
TSV : puzzle<TAB>solution

Example output

8 1 2 7 5 3 6 4 9		
9 4 3 6 8 2 1 7 5		
6 7 5 4 9 1 2 8 3		

1 5 4 2 3 7 8 9 6		
3 6 9 8 4 5 7 2 1		
2 8 7 1 6 9 5 3 4		

5 2 1 9 7 4 3 6 8		
4 3 8 5 2 6 9 1 7		
7 9 6 3 1 8 4 5 2		

Clues : 21
Valid : ✓ YES
Time : 314.73 µs (shared container — ~105 µs on dedicated HW)

Architecture

The solver is organized in 12 clearly separated sections:

Section	Content
1	Lookup tables (precomputed for AVX2 gather)
2	Grid state — BitBoard representation
3	Naked Singles (NS)
4	Hidden Singles with AVX2 gather
5	Propagation loop (NS + HS)

6	DFS helpers — entropy score, best block
7	Block-fill <code>fas()</code> — original technique
8	Smart dispatcher — 3-regime DFS
9	Input/output utilities
10	Interactive mode
11	File mode
12	<code>main()</code> entry point

Key innovations

1. Block-fill `fas()` — not documented in published solvers

Instead of branching cell by cell (standard DFS), `fas()` selects the most constrained 3×3 block and fills **all its empty cells simultaneously**, trying all valid combinations. When the block's candidate product is ≤ 48 , this reduces DFS depth significantly.

This technique was independently developed in 2026 and does not appear in any published Sudoku solver as of this writing.

2. AVX2 gather for Hidden Singles

For each row `r` and value `v`, `pcol[0..7]` and `pblk[block(r,0..7)]` are loaded with two `_mm256_i32gather_epi32` instructions, OR'd together, then compared with the value bitmask. This processes **8 cells in ~5 AVX2 instructions** instead of 8 scalar iterations. Applied identically to columns and blocks.

3. Smart 3-regime dispatcher

Different DFS strategies based on initial clue count:

- **≤ 16 clues** — entropy sort (orders candidates by constraint impact on neighbours)
 - **$= 17$ clues** — block-fill when best block score ≤ 48 , otherwise standard MRV
 - **≥ 18 clues** — entropy sort when a high-score move exists, otherwise block-fill
-

Potential for improvement — call for collaboration

Infandoku is currently **between jsolve (#3) and fsss2 (#6)** on the world ranking.
The path to reaching **#1 (tdoku)** is known — but not yet implemented here.

What tdoku does differently

tdoku uses a fundamentally different architecture:

instead of processing **1 grid at a time**, it packs **4 grids into a single AVX2 register** and processes all 4 simultaneously at every DFS node.

tdoku approach :

`cells[value][band] = __m256i` → 4 grids in 4 × 64-bit lanes

Every NS/HS/place operation touches 4 grids at once

At each DFS node : test 4 candidate values simultaneously

→ theoretical ×4 speedup on propagation

→ in practice : ×3 to ×6 vs single-grid solvers

What has been attempted

A multi-month development effort on V41 (4-grid AVX2 architecture) made significant progress:

-  4-grid encoding (`cells[v][band] = __m256i`) — validated
-  Vectorized NS on 4 grids simultaneously — validated
-  Vectorized HS on 4 grids simultaneously — validated
-  `place4()` — places different values in each of the 4 lanes, zero store/load — validated
-  4-way DFS — state management across 4 independent lanes without bugs — **not yet solved**

The core difficulty is managing grid state when lanes diverge in the DFS tree. tdoku solves this by maintaining lanes as **permanently interleaved** throughout the entire DFS, never repacking. Implementing this correctly without introducing state corruption bugs requires a complete rewrite of the DFS.

If you want to contribute

The foundation is solid. The missing piece is:

A DFS that broadcasts the current state into 4 lanes, injects 4 different candidate values

via `place4()` , propagates all 4 simultaneously, and correctly backtracks — **without ever accessing individual lanes outside of the initial broadcast and final solution extraction.**

If you understand how tdoku handles this and want to collaborate, contributions are very welcome.

The goal: bring Infandoku to **top 3 worldwide**.

Files

File	Description
<code>Infandoku.cpp</code>	Main solver — compile and run directly
<code>README.md</code>	This file

License

MIT License — free to use, modify, and publish.

If you use this code in academic work, please cite:

A. Infantine, "Infandoku — High-performance Sudoku Solver", 2026.
[https://github.com/\[your-username\]/infandoku](https://github.com/[your-username]/infandoku)
